

Influence Atlas — Data Extraction Playbook

Oct 3, 2026 · @Estrella

Purpose and status

This is the extraction reference for the Influence Atlas pipeline: for every source, where it lives, how to pull it, what comes back, and what breaks.

The graph we are reconstructing is **actor → ask → amendment → final article**. Every source below exists to populate one hop of that chain, or to resolve the actor behind it.

Verification status matters, so it is marked per source:

- **Confirmed** — documented on the provider's own pages or official release notes.
- **Third-party** — a community tool or aggregator, works but is not an official contract.
- **Unverified** — expected to exist, not checked. Probe it before building on it.

No endpoint here has been hit with a live request. Budget the first 15 minutes of the day to `curl` every base URL and record the actual response shape.

Design principles

Five rules. Breaking any of them costs hours later in the day.

1. **Raw before parsed.** Every fetch writes the untouched response to `raw/` first. Parsers read from disk, never from the network. If a parser breaks at 17:00 you re-run it in seconds instead of re-downloading 400 PDFs.
2. **Cache every HTTP call** by SHA-256 of the URL plus request body. A second identical request must never leave the machine.
3. **Manifest-driven.** A law enters the pipeline as one identifier (procedure id or CELEX). Step one resolves it into a `manifest.json` listing every derived id and which sources are available. Everything downstream reads the manifest, never a hardcoded value.
4. **Fail soft.** A missing source marks itself absent in the manifest and the run continues. A law with no public consultation must still produce a graph from amendments alone.
5. **Provenance on every row.** Source URL, fetch timestamp, and extraction method travel with the data. The jury will pick three edges at random and ask to see both original texts — those texts must be one lookup away.

Rate-limit yourself to roughly 2 requests per second per host and set a real `User-Agent` with a contact address. Getting blocked at 14:00 ends the day.

Layout and schemas

Global sources are fetched once. Per-law sources live under the procedure id, so adding a law never touches existing data.

```
data/
  raw/
    registry/           # Transparency Register dump
    meetings_ec/        # Commission lobby meetings
    meetings_mep/       # MEP lobby meetings
    meps/               # MEP roster
    laws/<proc_id>/
      manifest.json
    oeil/               # procedure timeline
    eurlex/            # proposal + final act XML
    hys/               # feedback JSON + attachment PDFs
    amendments/        # EP API JSON or committee PDFs
  parsed/
    actors.parquet
    meetings.parquet
    asks.parquet
    amendments.parquet
    articles.parquet
    links.parquet      # the graph edges
```

The five parsed tables and their keys:

Table	Grain	Key fields
actors	one organisation	actor_id (registry id), name, acronym, category, country, budget_eur, interests
meetings	one meeting	meeting_id, date, institution, host, actor_id, actor_raw_name, subject
asks	one text chunk from one submission	ask_id, law_id, actor_raw_name, actor_id, submitted_at, channel, text, source_url
amendments	one amendment	amendment_id, law_id, authors, target_unit, text_original, text_proposed, status
articles	one article or recital	law_id, unit_id, version, kind, text, changed_vs_proposal

Table	Grain	Key fields
links	one candidate edge	ask_id, amendment_id, unit_id, score, method, boilerplate_flag

Use Parquet, not CSV: legal text contains every delimiter you can think of.

Source A — EU Transparency Register

Confirmed. This is the master actor table. Every other source joins back to it.

Where. The register's own statistics page points to the complete list of registered organisations, downloadable in Open Government format from the European data portal (transparency-register.europa.eu). A full XML dump has existed since at least 2013, so expect XML with a CSV alternative.

How. One bulk download, no pagination, no key. Run it first — it is the single highest-value fetch of the day and it is independent of every law.

Fields to keep. Registration id, name, acronym, registration category (company, trade association, NGO, think tank, law firm), head office country, declared annual lobbying cost, number of accredited lobbyists, fields of interest, and the legislative files the organisation declares an interest in.

Parsing notes. The XML is deeply nested; flatten to one row per organisation and keep a separate long table for interests and declared files. Budget figures are declared as bands in some years and as exact values in others — normalise to a midpoint and keep the raw string.

Gotchas.

- The register is a snapshot, not a history. An organisation that deregistered is gone. For 2019 coverage you may need an archived copy or LobbyFacts (Source I).
- Self-declared budgets are known to be inconsistent. Treat spend as a weak signal — the challenge brief explicitly rewards finding winners who are *not* the biggest spenders.
- Some entries are individuals or one-person consultancies acting for undisclosed clients.

Source B — Commission lobby meetings

Confirmed. The cleanest relational data in the whole project, because it already carries the registry id.

Where. Published centrally in open data format since 2022 as the dataset [European Commission meetings with interest representatives](#) on data.europa.eu. Integrity Watch states it retrieves its Commission data from this portal.

How. Bulk download. Global, fetched once.

Fields to keep. Meeting date, host (Commissioner, cabinet member, or Director-General), portfolio, subject line, location, and the list of attending organisations with their registration ids.

Why it matters. The subject line is free text but usually names the policy file. That gives you a direct `actor → law` edge that needs no text matching at all — the most defensible link type you will have. Build it early.

Gotchas.

- Coverage is restricted to senior and management-level officials. Transparency International's own assessment is that this covers only a small share of actual lobbying, so absence of a meeting is never evidence of absence of lobbying.
- Subject lines are inconsistent: sometimes a procedure reference, sometimes a vague phrase like "digital policy". Plan a keyword and fuzzy match against law titles rather than an exact join.
- Pre-2022 meetings exist but came from separate per-Commissioner pages; expect format drift across the 2019–2022 window.

Source C — MEP lobby meetings

Confirmed, but messy. This is where lobbying meets the people who actually table amendments, so it is worth the pain.

Where. [Integrity Watch EU](#) covers MEP meetings from July 2019 onward, split by legislature. It states the data is scraped from MEPs' individual pages on the Parliament website. Downloads are offered, and the platform publishes its datasets as a matter of policy.

How. Take the Integrity Watch download rather than scraping 720 individual MEP pages yourself. Falling back to the EP pages is a half-day job on its own.

Fields to keep. Date, host MEP, political group, country, committee, role (rapporteur, shadow, member), subject, and the lobbyist organisation as typed.

The critical caveat. MEPs do not consistently log organisations under their official Transparency Register name — Transparency International reports only slightly over half follow that practice, which is why a clean Commission-style overview is not possible here. So:

- Never join this table on name alone. Route it through entity resolution (Source L) and keep `actor_raw_name` beside the resolved id.
- Publication is uneven across MEPs, so a silent MEP is not a clean MEP.

Why bother. A rapporteur or shadow rapporteur who met an organisation shortly before tabling an amendment that matches that organisation's submission is the single most persuasive edge you can show a jury. Store the meeting date so you can test ordering.

Source D — European Parliament Open Data Portal

Confirmed. API v2 has been available since the March 2024 release, which introduced new data models, EP taxonomies and version 2 of the API ([release notes](#)).

Where. `data.europarl.europa.eu`, Developer Corner. The portal also exposes DCAT dataset descriptions and bulk data dumps through the API, so check for a dump before paginating anything.

Datasets that matter here.

Dataset	What it gives you	Used for
<code>meps</code>	the roster, with group, country, dates	attributing amendments
<code>corporate-bodies</code>	committees and other EP bodies	mapping a law to its lead committee
<code>plenary-documents</code>	reports, motions and amendments tabled for plenary	Source H, primary route
<code>adopted-texts</code>	texts the Parliament adopted	the Parliament's position before trilogue
<code>meetings</code>	EP meeting events	timeline context

How. REST, JSON, no key expected. Fetch the MEP roster once globally; fetch documents per procedure.

Gotchas.

- The amendments exposed here are the *plenary* ones. Committee amendments, where most lobbying lands, are usually only in PDFs reached via OEIL. Plan for both routes.
- Documents often come as a metadata record plus a link to the actual file. The record is not the text.
- Parliament's open data rules commit to open licensing and machine-readable formats, so reuse is explicitly allowed — note the licence in your repo README.

Source E — Legislative Observatory (OEIL)

Unverified — probe first. OEIL (`oeil.secure.europarl.europa.eu`) is the spine of the per-law pipeline. Everything else is resolved from it.

What it gives. For one procedure reference such as `2021/0106(COD)` : the full timeline, the lead committee, the rapporteur and shadow rapporteurs, and links to every document produced at every stage — Commission proposal, committee report, amendments, Council position, adopted text.

How. There is no documented public API. Expect to fetch the procedure page and parse HTML, or check whether an RDF/XML export exists for that procedure. If OEIL fights back, the EP Open Data Portal (Source D) plus EUR-Lex procedure metadata (Source F) can reconstruct most of the same links.

This step produces the manifest. Write `raw/laws/<proc_id>/manifest.json` with:

```
{
  "procedure_id": "2021/0106(COD)",
  "title": "...",
  "celex_proposal": "52021PC0206",
  "celex_final": "32024R1689",
  "lead_committee": "IMCO-LIBE",
  "rapporteurs": [...],
  "hys_publication_id": 24212003,
  "documents": [{"type": "amendments", "url": "..."}],
  "available": {"hys": true, "amendments": true, "final_act": true}
}
```

That `available` block is the fail-soft switch. Everything downstream branches on it.

Gotchas. Procedure types other than COD (consent, own-initiative, delegated acts) have different document sets — do not assume a committee report exists. Non-COD procedures are where a demo of "any law" most often breaks.

Source F — EUR-Lex / Cellar

Confirmed. Cellar is the semantic database behind EUR-Lex. Two access routes, both public.

SPARQL at `http://publications.europa.eu/webapi/rdf/sparql` , with a [query builder](#) for composing queries. Use it to discover CELEX ids, titles, dates, EuroVoc subjects, and the relationship between a proposal and the act it became. The R package [eurlex](#) is the

best worked reference for query shapes even if you write Python — it has a documented function for retrieving the CELEX of a proposal of a given legal act, which is exactly the proposal-to-final mapping you need.

Direct retrieval by identifier:

```
GET http://publications.europa.eu/resource/celex/32024R1689
Accept: application/xml;notice=object
```

Swap the Accept header for the content you want; the same URL pattern serves metadata notices and document content. CELEX structure is documented in the [types of documents table](#) — sector 3 is legislation, 5 is preparatory acts.

Parsing — the part that earns points. Split the XML into one row per **article, paragraph and recital**, each with a stable unit id like `Art.6(2)(a)` or `Recital 27`. Granularity is the whole game: the jury reads "the final article" of an edge, so it must be a specific provision, not a 140-page act.

Then diff. Align proposal units against final-act units and flag which ones changed. Unchanged text cannot be evidence of influence, so this diff cuts your candidate space by a large factor and sharpens every downstream match.

Gotchas. Fetch one language (English) and stay there. Article numbering shifts between proposal and final act, so align on text similarity plus position, not on number. Rate-limit: Cellar is slow and will throttle a tight loop.

Source G — Have Your Say

Third-party route, confirmed to work. This is the "what they ask" side of the comparison, and the richest text you will get.

Where. The Commission's [Have Your Say](#) platform. No official API found.

Getting the id. Every initiative page URL ends in a numeric **publication id**. For the AI Act Commission-adoption initiative that id is `24212003`. Resolve it per law and store it in the manifest.

How. The [hys_scraper](#) package on PyPI scrapes an initiative's feedback submissions, the attachments of those submissions, and by-country and by-category statistics:

```
pip install hys_scraper
python -m hys_scraper 24212003
```

It writes an `attachments/` folder plus `attachments.csv` and `feedback/statistics` frames, and can also be called in Python as `HYS_Scraper("24212003").scrape()`. Read its source to learn the underlying endpoints, then call them directly if you need more control — that

is also your fallback if the package is stale.

Chunking — do not skip this. A position paper carries thirty distinct asks across twenty pages. Treat the whole PDF as one unit and your matching is worthless. Split attachments into paragraphs or numbered sections, one `ask` row each, keeping page number and offset so you can show the jury the exact passage.

Fields to keep. Organisation name as typed, user type (company, NGO, citizen, public authority), country, submission date, the free-text feedback field, and every attachment with extracted text.

Gotchas.

- Separate organisations from private citizens early; citizen submissions are numerous and will swamp any ranking.
- Some attachments are scanned images. Either run OCR or mark them unreadable and exclude them from scoring — never silently drop them.
- Attachments appear in several languages. Either translate or restrict matching to English submissions and declare that limit in the report.
- Not every law has a consultation with attachments. The manifest's `available.hys` flag must handle that.

Source H — Amendments

The weakest link in the chain. Allocate your strongest engineer here and start early.

Route 1 — EP API (preferred). The `plenary-documents` dataset lists texts tabled for plenary: reports, motions for resolutions, amendments and other documents proposed for formal discussion. Structured and fast, but plenary-only.

Route 2 — committee PDFs (where the real lobbying is). Reach them through the OEIL document links. Committee amendment documents follow a rigid repeating layout:

```
Amendment 147
José Example, Another MEP

Proposal for a regulation
Article 6 - paragraph 2 - point a

| Text proposed by the Commission | Amendment |
```

That two-column before/after table is a gift: it hands you the original text, the proposed text and the exact target unit in one block. Write a regex parser over the "Amendment N" header plus the two-column body, and validate it by hand against two documents from different committees before trusting it.

Fields to keep. Amendment number, author MEPs (resolve against the roster from Source D), committee, target unit as written, text proposed by the Commission, amendment text, and justification where present. The justification field is unusually valuable: it sometimes paraphrases a lobbying argument almost verbatim.

Adoption status — infer it, don't hunt for it. Rather than chasing voting records, compare each amendment's proposed text against the final act. If the wording survives, it was adopted. This is more robust than roll-call parsing and it reuses the article table you already built. Record `status` as adopted / not-found / partial, with the matched unit id.

Gotchas.

- Amendment numbering restarts per document; the unique key is (document id, amendment number).
- Some amendments are tabled by a whole political group rather than named MEPs.
- Compromise amendments merge several originals and often carry no author at all — flag them separately rather than discarding them.
- Trilogue is not public. The final jump from Parliament position to final act is partly invisible; say so in the report instead of pretending the chain is complete.

Sources I-M — enrichment, only if time allows

None of these is on the critical path. Pull them only once a full law runs end to end.

#	Source	Gives you	Status	Access
I	LobbyFacts.eu	historical register snapshots, so 2019-era budgets and deregistered bodies	Unverified	bulk download, check last update
J	Council document register	Council positions and compromise texts, the missing trilogue side	Unverified	search + PDF scrape
K	Organisation websites	position papers, press releases — the "what they say in public" half	Build yourself	targeted crawl of the top 50 actors, plus Wayback for older pages
L	GDELT	news coverage volume and framing per actor and topic	Confirmed to exist	BigQuery public dataset or file downloads

#	Source	Gives you	Status	Access
M	Wikidata · GLEIF	canonical ids and aliases for companies	Confirmed	SPARQL / bulk LEI file

Going global (ambition points, 15 on the scorecard): US lobbying disclosures at `lda.senate.gov` expose a REST API; `regulations.gov` is the US equivalent of Have Your Say; Germany's Lobbyregister, France's HATVP and Ireland's lobbying.ie publish open data. Finland even runs an auth-free JSON Transparency Register API, which is a cheap proof that the schema generalises beyond the EU.

Scope discipline. Each of these is a half-day on its own. Pulling one shallow extra country beats pulling five badly, and a report that says which sources were deliberately left out reads as rigour, not as a gap.

Entity resolution

Three populations of names have to collapse into one actor id, and only one of them is clean.

Population	Name quality	Join key available
Transparency Register (Source A)	canonical	registration id
Commission meetings (Source B)	canonical	registration id, direct
MEP meetings (Source C)	free text, inconsistent	none
Have Your Say submissions (Source G)	free text, self-typed	none

Layered strategy, most reliable first. Stop at the first layer that matches and record which one fired.

1. **Exact registration id.** Covers Commission meetings outright.
2. **Normalised exact match.** Lowercase, strip legal suffixes (GmbH, SA, AISBL, e.V., Ltd, BV), strip punctuation and "the", collapse whitespace. Match against name and acronym.
3. **Acronym match.** Many submissions use only the acronym (`BEUC` , `CCIA`). Build an acronym index from the register.
4. **Fuzzy match** on normalised names with a high threshold (token-set ratio above roughly 90). Below that, leave unresolved rather than guess.

5. **Manual review of the top 50** unresolved names by frequency. Half an hour of eyeballing fixes most of the remaining noise, because the distribution has a very long head.

Always store, per row: `actor_raw_name`, `actor_id` (nullable), `match_method`, `match_score`. Unresolved is a legitimate state — a visible "unresolved" count is more credible than a silently wrong join, and the jury can ask.

Known traps. Corporate groups versus national subsidiaries (one parent, twelve entries). Trade associations whose members are themselves registered, which double-counts influence unless you model membership. Law firms and consultancies filing on behalf of undisclosed clients — the registered actor is not the interested party, and saying so in the report is itself an insight.

Validation and the generalisation test

Do not start matching until extraction passes these three gates.

Gate 1 — parser spot-checks. Twenty random rows from each parser, read by a human against the source document. Article splitting, amendment parsing and PDF text extraction each fail silently and each will produce confident nonsense.

Gate 2 — the three-law test, by 13:00. Run the full extraction on three deliberately different laws:

Law type	Why it is in the set
Large, high-profile COD (e.g. AI Act)	plenty of material to debug against
Small, low-salience COD	tests behaviour when counts are tiny
Non-COD, or one with no public consultation	tests the fail-soft path

The large law is a development fixture, not a representative sample. Whatever breaks on laws two and three is exactly what breaks when the jury names a law on stage.

Gate 3 — coverage report per law. Auto-generate it, and show it in the demo:

- sources found versus sources missing
- counts: submissions, organisations, amendments, articles changed
- share of actor names resolved, by match method
- date range actually covered

Showing "here is what exists and here is what cannot be known for this law" is stronger than a confident graph with silent holes. It directly serves the 25-point real-links criterion, because it tells the jury where your edges are solid.

Timebox. If extraction for a given law is not done by its slot, mark the law partial and move on. A pipeline that handles ten laws at 70% beats one that handles one at 95%.

Risks and mitigations

Risk	Why it bites	Mitigation
OEIL has no API	the per-law spine depends on it	probe at 09:30; fall back to EP Open Data + EUR-Lex procedure metadata
Committee amendments only in PDF	the richest amendment layer is unstructured	regex on the two-column layout, hand-validated on two documents
Boilerplate false positives	shared legal phrasing links everything to everything	drop high-frequency n-grams; require the target unit to have changed vs the proposal
Trilogue is opaque	the amendment → final article hop is partly unobservable	infer adoption by text survival; state the limit explicitly in the report
Scanned or non-English attachments	silent data loss	OCR or mark unreadable; never drop without counting
MEP meeting names don't match the register	joins look fine and are wrong	resolve through the layered matcher, keep the raw name
Rate limiting or an IP block	ends the day	cache everything, 2 req/s, real User-Agent
Register is a snapshot	2019 actors are missing	LobbyFacts or Wayback if time allows; otherwise scope the claim to current registrants
Over-fitting to one law	the live "any law" demo fails	the three-law test at 13:00, non-negotiable

The failure mode that actually loses the day is spending until 17:00 on a beautiful AI Act graph and discovering at 19:00 that nothing else runs. Freeze features at 15:00 and spend the rest on breadth plus the report.

Execution order

Global sources are independent of everything and give the most data per unit of effort — launch them in the first twenty minutes.

Slot	A	B	C	D
09:30–11:00	A, B, C, D (global pulls)	G: Have Your Say scrape	E, F: OEIL + Cellar	repo scaffold, HTTP cache, manifest schema
11:00–13:00	L: entity resolution	PDF text extraction + chunking	H: amendments	proposal → final diff
13:00–14:00	three-law test, everyone			
14:00–17:00	matching + boilerplate filter	graph assembly	coverage report	repo, README, licence
17:00–19:00	insight hunting, forecast		demo rehearsal with a law nobody prepared	report writing

Dependencies. G and H can start the moment E produces a manifest. L needs A and C. Nothing needs the matcher, which is why the matcher is not in the morning.

Expect the jam at amendments. Whoever finishes a global pull first joins C on Source H rather than polishing their own.

Hard stops. Three-law test at 13:00. Feature freeze at 15:00. Repo public with an open licence by 18:00 — the brief requires it, and it is 15 points you can bank while others are still debugging.